

Age prediction: data preparation and explanatory data analysis (EDA)

Import libraries

```
In [ ]: import numpy as np #library for tools for calculations
import pandas as pd #library for data manipulation
import matplotlib.pyplot as plt #library for plots
```

Loading and showing input (features) and output data (desired output)

```
In [ ]: df = pd.read_excel('https://raw.githubusercontent.com/sdiciotti/Age-Prediction-
Demo/main/NKI2_data.xlsx') #takes data from the xlsx pointed and puts them in df
print (type(df)) #print the type of df (<class 'libreria di origine modulo di
Pandas=.type effettivo>)
```

```
<class 'pandas.core.frame.DataFrame'>
```

Print the size of the dataframe

```
In [ ]: df.shape #dimensions of our df (it returns a tuple)
```

```
(73, 35)
```

Print the dataframe

```
In [ ]: print(df) #print the whole df
```

Check the features and output variables name

```
In [ ]: features = list(df.columns) #takes the columns name and using the list function it
returns a list with the columns names
type(features) #just checks the features type
```

```
list
```

```
In [ ]: print(features) #print all features
```

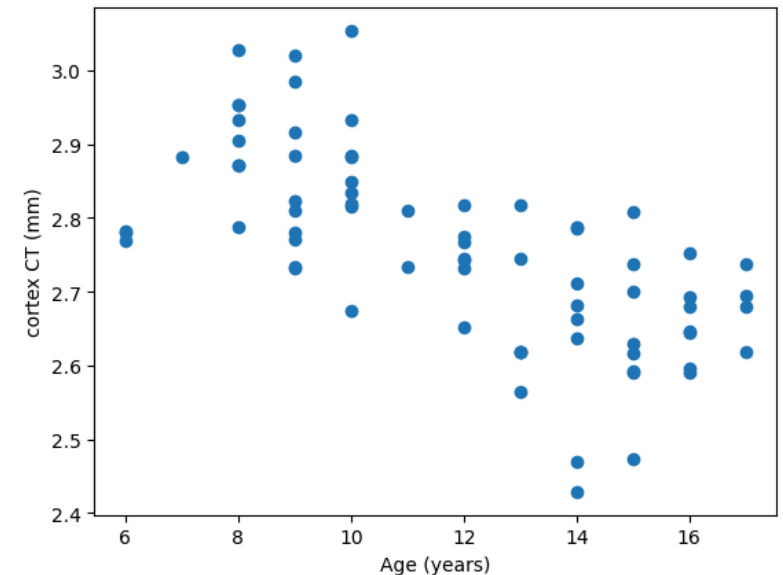
```
['Age', 'Sex', 'cortex_CT', 'lh_cortex_CT', 'rh_cortex_CT',
'lh_frontalGM_CT', 'lh_occipitalGM_CT', 'lh_temporalGM_CT',
```

```
'lh_parietalGM_CT', 'rh_frontalGM_CT', 'rh_occipitalGM_CT',
'rh_temporalGM_CT', 'rh_parietalGM_CT', 'cortex_pial_lgi',
'lh_cortex_pial_lgi', 'rh_cortex_pial_lgi', 'lh_frontal_pial_lgi',
'lh_occipital_pial_lgi', 'lh_temporal_pial_lgi', 'lh_parietal_pial_lgi',
'rh_frontal_pial_lgi', 'rh_occipital_pial_lgi', 'rh_temporal_pial_lgi',
'rh_parietal_pial_lgi', 'cerebralGM_FD', 'lh_cerebralGM_FD',
'lh_frontalGM_FD', 'lh_temporalGM_FD', 'lh_parietalGM_FD',
'lh_occipitalGM_FD', 'rh_cerebralGM_FD', 'rh_frontalGM_FD',
'rh_temporalGM_FD', 'rh_parietalGM_FD', 'rh_occipitalGM_FD']
```

Plotting some data

```
In [ ]: plt.scatter(df['Age'],df['cortex_CT']) #print a scatter plot with age data on x
and cortex_CT on y
plt.xlabel('Age (years)') #xlabel
plt.ylabel('cortex CT (mm)') #ylabel
```

```
Text(0, 0.5, 'cortex CT (mm)')
```



```
In [ ]: print("Dataframe shape before NaN removal:", df.shape) #just reprints the
dimensions before data manipulation
```

```
Dataframe shape before NaN removal: (73, 35)
```

Removing missing data

```
In [ ]: df.dropna(axis=0, how='any', inplace=True) #start analyzing all rows (axis = 0),
if ANY of the values on that row is NaN then delete the row, True indicates that
the modifications are written on df directly
```

```
print("Dataframe shape after NaN removal:", df.shape) #new dimensions (there was 1 row with NaN)
```

Dataframe shape after NaN removal: (72, 35)

Preparing the design matrix X and the desired output y

```
In [ ]: X = df.iloc[:,2:] #integer location, take all the rows and all the columns from the 2 on and place them in X
y = df['Age'] #extract a single column from df and place it in y
print (type(X))
print (type(y))
print (X.shape)
print (y.shape)
```

```
<class 'pandas.core.frame.DataFrame'>
<class 'pandas.core.series.Series'>
(72, 33)
(72,)
```

A quick quality control

```
In [ ]: X.head(6) #shows the first 6 rows of X
```

```
In [ ]: print('The whole dataset contains ' + str(X.shape[0]) + ' subjects') #need to convert an integer into string to put it in a print like this
print('The age prediction will be performed using ' + str(X.shape[1]) + ' MRI-derived features') #same
```

```
The whole dataset contains 72 subjects
The age prediction will be performed using 33 MRI-derived features
```

Underfitting and Overfitting – Interactive Demo

In this demo, we use polynomial regression to explore the concepts of **underfitting** and **overfitting**

- The true function we want to learn is a simple **quadratic** curve.
- We generate noisy observations of that curve.
- You can interactively adjust:
- The **degree** of the polynomial model (its capacity)
- The **noise level** added to the data

Use the sliders to observe how model complexity and noise affect both **training** and **test error**.

```
In [ ]: # Install ipywidgets if not already available
!pip install ipywidgets --quiet

from ipywidgets import interact #interact is used to create interactive sliders
import numpy as np
import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression #linear model used
from sklearn.preprocessing import PolynomialFeatures #transforms linear variables
in polynomial ones
from sklearn.metrics import mean_squared_error #our performance metric
from sklearn.model_selection import train_test_split #function used to divide
dataset in training set e test set

# Base data (same for all runs)
np.random.seed(0) #stops the random generator so its replicable
n_samples = 100 #define the number of samples
X = np.linspace(0, 1, n_samples).reshape(-1, 1) #first it creates 100 equispaced
points between 0 and 1 and then transforms the vector in a 1 column matrix
# y_true = np.sin(2 * np.pi * X).ravel()
y_true = 4 * (X.ravel() - 0.5) ** 2 #ravel transforms X in a monodimensional
vector and then its used to generate a perfect U parabola

# Interactive plot function (every time we use the slider)
def plot_polynomial_fit(degree=1, noise_std=0.1):
    # Generate noisy labels
    y = y_true + noise_std * np.random.randn(n_samples) #create real values adding
noise

    # Train/test split
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42) #divide in training and test set

    # Polynomial features
```

```
poly = PolynomialFeatures(degree) #if degree = 3 this block takes x and
returns [1,x,x^2,x^3]
X_train_poly = poly.fit_transform(X_train) #fit implies the study of the data
while tranform does the transformation
X_test_poly = poly.transform(X_test) #just use the transformation based on the
parameters obtained form training
#if we were to use fit_ecc on the test we could risk data leakage, polluting
the model with validation data

# Train model
model = LinearRegression() #initialize the model
model.fit(X_train_poly, y_train) #trains the model on the training set y_train
and finds the optimal parameters

# Predictions and errors
y_train_pred = model.predict(X_train_poly) #prediction on training set
y_test_pred = model.predict(X_test_poly) #prediction on test set
train_mse = mean_squared_error(y_train, y_train_pred) #mse on training
test_mse = mean_squared_error(y_test, y_test_pred) #mse on test

# Plotting
X_plot = np.linspace(0, 1, 200).reshape(-1, 1) #create a dense line to have a
continuous plot
y_plot = model.predict(poly.transform(X_plot))

plt.figure(figsize=(7, 5))
plt.scatter(X_train, y_train, color='blue', label='Train data', s=20)
plt.plot(X_plot, y_plot, color='red', label=f'Degree {degree}')
plt.plot(X, y_true, '--', color='green', label='True function')
plt.title(f"Train MSE = {train_mse:.4f} | Test MSE = {test_mse:.4f} | Noise
std = {noise_std}")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid(True)
plt.show()

# Activate interactive widgets
interact(plot_polynomial_fit, degree=(1, 15), noise_std=(0.0, 1.0, 0.05))
```

```
interactive(children=(IntSlider(value=1, description='degree', max=15,
min=1), FloatSlider(value=0.1, descript...
```

```
plot_polynomial_fit
def plot_polynomial_fit(degree=1, noise_std=0.1)
```

```
<no docstring>
```

 MSE vs Polynomial Degree

This plot shows how the **training** and **test Mean Squared Error (MSE)** change as we increase the degree of the polynomial model.

- We compare two scenarios:
- Low noise (noise_std = 0.15)
- High noise (noise_std = 0.45)

Key insights:

- Low-degree models may underfit (high error on both sets)
- High-degree models may overfit (low training error but high test error)
- Higher noise levels make overfitting occur earlier

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split

# Base data
np.random.seed(0)
n_samples = 100
X = np.linspace(0, 1, n_samples).reshape(-1, 1)
#y_true = np.sin(2 * np.pi * X).ravel()
y_true = 4 * (X.ravel() - 0.5) ** 2
degrees = range(1, 16) #defines the degrees to test, we make them change
automatically
noise_levels = [0.15, 0.45] #low noise scenario and high noise scenario

plt.figure(figsize=(10, 5)) #initializes the figure

for noise_std in noise_levels: #firs 0.15 then 0.45
    y = y_true + noise_std * np.random.randn(n_samples) #takes the noise from
    levels and generates the dataset
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    random_state=42) #divide in training and test set
    train_mse = [] #empty list at each noise change, used to store the training
    errors
    test_mse = [] #empty list at each noise change, used to store the test errors

    for degree in degrees: #for the present noise level it trains 15 different
    models (degrees)
        poly = PolynomialFeatures(degree) #initializes the transformation tool
        X_train_poly = poly.fit_transform(X_train) #study the training set
        X_test_poly = poly.transform(X_test) #just transformation

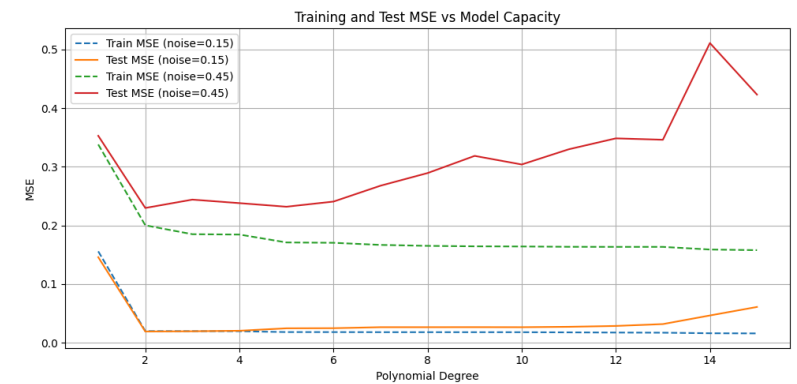
        model = LinearRegression() #initializes the model
        model.fit(X_train_poly, y_train) #training of the model, find the weights
        coefficients that minimize the distance from the training points

        y_train_pred = model.predict(X_train_poly) #asks for a prediction (stima)
        using training data
        y_test_pred = model.predict(X_test_poly) #idem but with test data (never
        seen during training)
```

```
train_mse.append(mean_squared_error(y_train, y_train_pred)) #save the MSEs
on the empty vectors
test_mse.append(mean_squared_error(y_test, y_test_pred))

# Plot for current noise level
plt.plot(degrees, train_mse, linestyle='--', label=f'Train MSE (noise=
{noise_std})')
plt.plot(degrees, test_mse, label=f'Test MSE (noise={noise_std})')

plt.xlabel('Polynomial Degree')
plt.ylabel('MSE')
plt.title('Training and Test MSE vs Model Capacity')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



Age Prediction: data preparation, Exploratory Data Analysis (EDA), and regression with hold-out & nested hold-out validation

Data Preparation and Exploratory Data Analysis (EDA)

Import libraries

```
In [ ]: # Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.svm import SVR #we'll need it at the end
```

Loading and showing input (features) and output data (desired output)

```
In [ ]: df = pd.read_excel('https://raw.githubusercontent.com/sdiciotti/Age-Prediction-
Demo/main/NKI2_data.xlsx')
print (type(df))
```

```
<class 'pandas.core.frame.DataFrame'>
```

Print the size of the dataframe

```
In [ ]: df.shape
```

```
(73, 35)
```

Print the dataframe

```
In [ ]: print(df)
```

Check the features and output variables name

```
In [ ]: features = list(df.columns)
type(features)
```

```
list
```

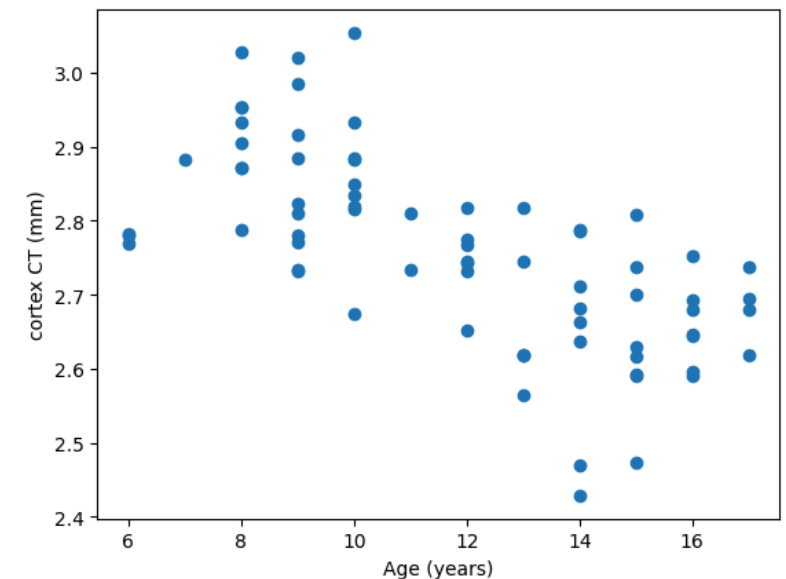
```
In [ ]: print(features)
```

```
['Age', 'Sex', 'cortex_CT', 'lh_cortex_CT', 'rh_cortex_CT',
'lh_frontalGM_CT', 'lh_occipitalGM_CT', 'lh_temporalGM_CT',
'lh_parietalGM_CT', 'rh_frontalGM_CT', 'rh_occipitalGM_CT',
'rh_temporalGM_CT', 'rh_parietalGM_CT', 'cortex_pial_lgi',
'lh_cortex_pial_lgi', 'rh_cortex_pial_lgi', 'lh_frontal_pial_lgi',
'lh_occipital_pial_lgi', 'lh_temporal_pial_lgi', 'lh_parietal_pial_lgi',
'rh_frontal_pial_lgi', 'rh_occipital_pial_lgi', 'rh_temporal_pial_lgi',
'rh_parietal_pial_lgi', 'cerebralGM_FD', 'lh_cerebralGM_FD',
'lh_frontalGM_FD', 'lh_temporalGM_FD', 'lh_parietalGM_FD',
'lh_occipitalGM_FD', 'rh_cerebralGM_FD', 'rh_frontalGM_FD',
'rh_temporalGM_FD', 'rh_parietalGM_FD', 'rh_occipitalGM_FD']
```

Plotting some data

```
In [ ]: # Plot true vs predicted values to visualize model performance
plt.scatter(df['Age'],df['cortex_CT'])
plt.xlabel('Age (years)')
plt.ylabel('cortex CT (mm)')
```

```
Text(0, 0.5, 'cortex CT (mm)')
```



```
In [ ]: print("Dataframe shape before NaN removal:", df.shape)
```

```
Dataframe shape before NaN removal: (73, 35)
```

Removing missing data

```
In [ ]: df.dropna(axis=0, how='any', inplace=True)
        print("Dataframe shape after NaN removal:", df.shape)
```

Dataframe shape after NaN removal: (72, 35)

Preparing the design matrix X and the desired output y

```
In [ ]: X = df.iloc[:,2::]
        y = df['Age']
        print (type(X))
        print (type(y))
        print (X.shape)
        print (y.shape)

<class 'pandas.core.frame.DataFrame'>
<class 'pandas.core.series.Series'>
(72, 33)
(72,)
```

A quick quality control

```
In [ ]: X.head(6)
```

```
In [ ]: print('The whole dataset contains ' + str(X.shape[0]) + ' subjects')
        print('The age prediction will be performed using ' + str(X.shape[1]) + ' MRI-
        derived features')
```

The whole dataset contains 72 subjects
The age prediction will be performed using 33 MRI-derived features

Regression task

Linear Regression using a hold-out validation scheme

```
In [ ]: from sklearn.model_selection import train_test_split
        from sklearn.linear_model import LinearRegression
        from sklearn.metrics import mean_absolute_error

        # Setting the seed of the random generator
        SEED = 42 #reference to The Hitchiker's Guide to the Galaxy, we need to set it to
        have the same pseudo-random values each time
```

```
# Size of the samples in the test set: e.g., 0.1, means that the test set is
composed of 10% of the samples of the entire dataset
test_size = 0.1 #10% of the data will be test set
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=test_size,
random_state=SEED) #does the separation
```

```
In [ ]: type(X_train) #asks for the type of the matrix generated (DataFrame)
```

pandas.core.frame.DataFrame
def __init__(data=None, index: Axes | None=None, columns: Axes | None=None, dtype: Dtype | None=None, copy: bool | None=None) -> None

Two-dimensional, size-mutable, potentially heterogeneous tabular data.

Data structure also contains labeled axes (rows and columns). Arithmetic operations align on both row and column labels. Can be thought of as a dict-like container for Series objects. The primary

```
In [ ]: type(y_train) #= (Series)
```

pandas.core.series.Series
def __init__(data=None, index=None, dtype: Dtype | None=None, name=None, copy: bool | None=None, fastpath: bool | lib.NoDefault=lib.no_default) -> None

One-dimensional ndarray with axis labels (including time series).

Labels need not be unique but must be a hashable type. The object supports both integer- and label-based indexing and provides a host of methods for performing operations involving the index. Statistical

```
In [ ]: # Print the shape of the training and test sets
        print (X_train.shape, y_train.shape)
        print (X_test.shape, y_test.shape)
```

(64, 33) (64,)
(8, 33) (8,)

```
In [ ]: X_train.head(2) #return the first 2 rows of X_train
```

```
In [ ]: y_train.head(2)
```

```
In [ ]: # Calculate the Mean Absolute Error (MAE) to evaluate model performance
        # Fitting the linear model
        reg = LinearRegression().fit(X_train, y_train) #evaluate the best coefficients and
        returns the mathematical formula based on training set

        # Applying the final model to the test set samples
        y_pred = reg.predict(X_test) #takes the model implemented and gives it the test
        set, returns predictions
        #in this string we dont use y_test still but we make blind predictions based on
```

```

the training

# Compute the MAE in the test set
MAE = mean_absolute_error(y_test, y_pred) #evaluates the MAE of the model on the
test set
#it takes the real value (y_test) and the predicted one (y_pred) and calculate the
MAE
print ("MAE test set:", MAE, "years")

```

MAE test set: 1.4109915646679987 years

```

In [ ]: type(y_pred)

numpy.ndarray

```

Polynomial regression using a nested hold-out validation scheme

```

In [ ]: from sklearn.model_selection import train_test_split
#here we perform a double splitting

# Setting the seed of the random generator
SEED = 42

# Size of the samples in the test set: e.g., 0.1, means that the test set is
composed of 10% of the samples of the entire dataset
test_size = 0.1

# Size of the samples in the validation set: e.g., 0.1, means that the test set is
composed of 10% of the samples of the development set
val_size = 0.1 #validation set will be 10%

# Outer hold-out (development and test)
X_dev, X_test, y_dev, y_test = train_test_split(X, y, test_size=test_size,
random_state=SEED)

# Print the size of the development and test sets
print ("The development set size is", "X:", X_dev.shape, "y:", y_dev.shape)
print ("The test set size is", "X:", X_test.shape, "y:", y_test.shape)

# Inner hold-out (training and validation)
X_train, X_val, y_train, y_val = train_test_split(X_dev, y_dev,
test_size=val_size, random_state=SEED)
# Print the size of the training and validation sets
print ("\nThe development set size is", "X:", X_train.shape, "y:", y_train.shape)
print ("The test set size is", "X:", X_val.shape, "y:", y_val.shape)
print()

```

The development set size is X: (64, 33) y: (64,)
The test set size is X: (8, 33) y: (8,)

The development set size is X: (57, 33) y: (57,)

The test set size is X: (7, 33) y: (7,)

```

In [ ]: from sklearn.preprocessing import PolynomialFeatures

MAE = []
degree_list = [1, 2, 3, 4, 5] # List of polynomial degrees to test

for d in degree_list:
    poly = PolynomialFeatures(degree=d) #configure the transformation using the
actual degree
    X_train_poly = poly.fit_transform(X_train) #study the training set structure
and determine the geometry (x^2, x^n ecc.)
    X_val_poly = poly.transform(X_val) #use the same transformation on the
validation set (no fit)

    reg = LinearRegression().fit(X_train_poly, y_train) #create the model and
train it calculating the weights
    y_val_pred = reg.predict(X_val_poly) #obtain predictions using the weights

    MAE_d = mean_absolute_error(y_val, y_val_pred) #compute the MAE for the order
d on the validation set
    print("**** Polynomial degree:", d, "MAE:", MAE_d)
    MAE.append(MAE_d) #memorize the MAE

# Find the best degree with the lowest MAE
best_degree_index = np.argmin(MAE) #index at which we find the lowest MAE
best_degree = degree_list[best_degree_index] #value of the lowest MAE
print("\nBest polynomial degree:", best_degree, "MAE:", MAE[best_degree_index],
"years")

# Re-train the Polynomial Regression model on the development set using the best
degree
poly = PolynomialFeatures(degree=best_degree) #impost a new polynomial using the
best degree just found
X_dev_poly = poly.fit_transform(X_dev) #transform the whole developmente set for
final training
X_test_poly = poly.transform(X_test) #transform the test set using the same
transformation

reg = LinearRegression().fit(X_dev_poly, y_dev) #train and compute the final
weights on the whole development set

# Apply the model to the test set and compute the MAE
y_test_pred = reg.predict(X_test_poly) #generates the final predictions on the
test set
MAE_test = mean_absolute_error(y_test, y_test_pred) #compute the final MAE
print("MAE test set:", MAE_test, "years")

# Plot predictions in the test set
plt.scatter(y_test, y_test_pred)
plt.xlabel('True y')
plt.ylabel('Predicted y')
plt.title(f'Polynomial Regression (Degree {best_degree})')
plt.show()

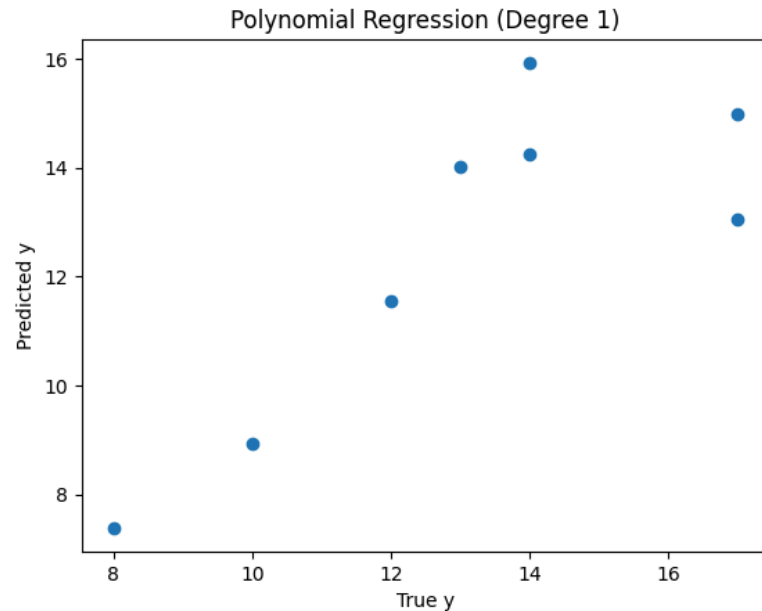
```

```

*** Polynomial degree: 1 MAE: 1.529087405773741
*** Polynomial degree: 2 MAE: 2.0430609668447874
*** Polynomial degree: 3 MAE: 2.019123282529224
*** Polynomial degree: 4 MAE: 2.002122399598142
*** Polynomial degree: 5 MAE: 2.003922839417611

```

Best polynomial degree: 1 MAE: 1.529087405773741 years
MAE test set: 1.4109915646680058 years



Support Vector Regressor with hyperparameter C (Complexity) using a nested hold-out validation scheme

```

In [ ]: MAE = []
C_list = [0.1, 1, 10] #candidates as hyperparameters
for c in C_list: #tries every C
    reg = SVR(kernel='rbf', degree=3, gamma='scale', coef0=0.0, tol=0.001, C=c,
epsilon=0.1, shrinking=True, cache_size=200, verbose=0, max_iter=- 1).fit(X_train,
y_train)
    #initliaze the Support Vector Regressor model with the parameters listed, then
    .fit trains it on training set computing weights and optimal support vectors
    y_val_pred = reg.predict(X_val) #generates predictions for the validation set
    MAE_c = mean_absolute_error(y_val, y_val_pred) #compute MAE relative to that C
    print ("*** C value:", c, "MAE:", MAE_c)
    MAE.append (MAE_c) #store the value
bestC_index = np.argmin(MAE) #index of the best MAE in the MAE vector
bestC = C_list[bestC_index] #value of the best MAE
print ("\nBest C: ", bestC, "MAE", MAE[bestC_index], "years")

```

```

# Re-train the SVR model in the development test using the bestC hyperparameter
reg = SVR(kernel='rbf', degree=3, gamma='scale', coef0=0.0, tol=0.001, C=bestC,
epsilon=0.1, shrinking=True, cache_size=200, verbose=0, max_iter=- 1).fit(X_dev,
y_dev)

```

```

# Apply the model to the test set and compute the MAE
y_test_pred = reg.predict(X_test) #compute predictions on the test set
MAE = mean_absolute_error(y_test, y_test_pred) #compute and print final MAE
print ("MAE test set:", MAE, "years")

```

```

# Plot predictions in the test set
plt.scatter(y_test,y_test_pred)
plt.xlabel('true y')
plt.ylabel('predicted y')

```

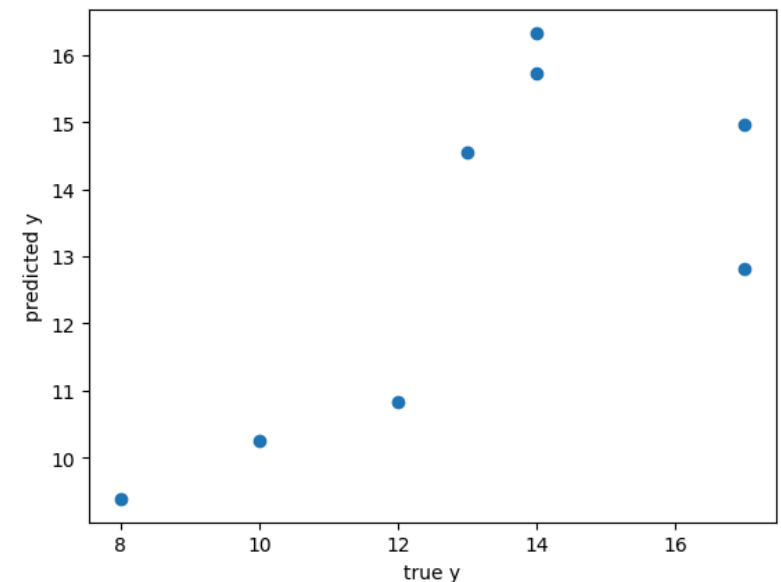
```

*** C value: 0.1 MAE: 2.744695146993926
*** C value: 1 MAE: 2.157673642649064
*** C value: 10 MAE: 1.7345360725342982

```

Best C: 10 MAE 1.7345360725342982 years
MAE test set: 1.8307604143733762 years

```
Text(0, 0.5, 'predicted y')
```



Age Prediction: data preparation, Exploratory Data Analysis (EDA), and classification with hold-out & nested hold-out

Data Preparation and Exploratory Data Analysis (EDA)

Import libraries

```
In [ ]: # Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Loading and showing input (features) and output data (desired output)

```
In [ ]: df = pd.read_excel('https://raw.githubusercontent.com/sdiciotti/Age-Prediction-
Demo/main/NKI2_data.xlsx')
print (type(df))
```

```
<class 'pandas.core.frame.DataFrame'>
```

Print the size of the dataframe

```
In [ ]: df.shape
```

```
(73, 35)
```

Print the dataframe

```
In [ ]: print(df)
```

Check the features and output variables name

```
In [ ]: features = list(df.columns)
type(features)
```

```
list
```

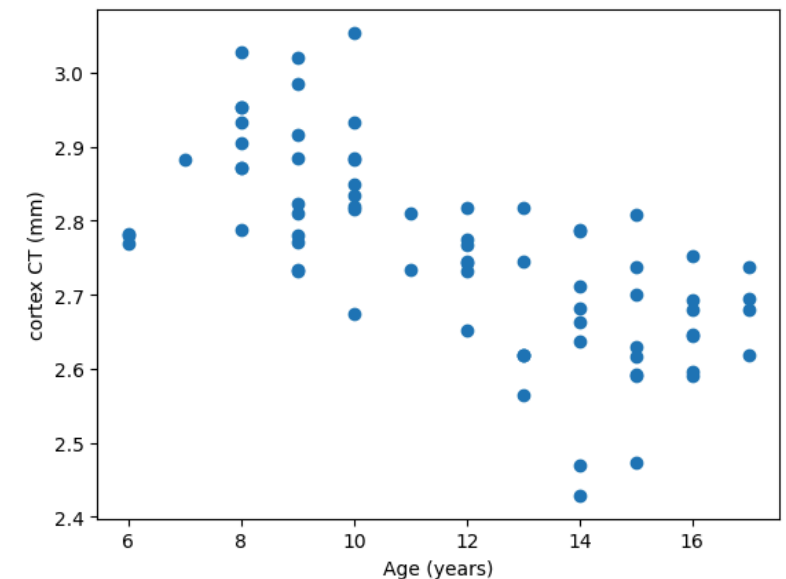
```
In [ ]: print(features)
```

```
['Age', 'Sex', 'cortex_CT', 'lh_cortex_CT', 'rh_cortex_CT',
'lh_frontalGM_CT', 'lh_occipitalGM_CT', 'lh_temporalGM_CT',
'lh_parietalGM_CT', 'rh_frontalGM_CT', 'rh_occipitalGM_CT',
'rh_temporalGM_CT', 'rh_parietalGM_CT', 'cortex_pial_lgi',
'lh_cortex_pial_lgi', 'rh_cortex_pial_lgi', 'lh_frontal_pial_lgi',
'lh_occipital_pial_lgi', 'lh_temporal_pial_lgi', 'lh_parietal_pial_lgi',
'rh_frontal_pial_lgi', 'rh_occipital_pial_lgi', 'rh_temporal_pial_lgi',
'rh_parietal_pial_lgi', 'cerebralGM_FD', 'lh_cerebralGM_FD',
'lh_frontalGM_FD', 'lh_temporalGM_FD', 'lh_parietalGM_FD',
'lh_occipitalGM_FD', 'rh_cerebralGM_FD', 'rh_frontalGM_FD',
'rh_temporalGM_FD', 'rh_parietalGM_FD', 'rh_occipitalGM_FD']
```

Plotting some data

```
In [ ]: # Plot true vs predicted values to visualize model performance
plt.scatter(df['Age'],df['cortex_CT'])
plt.xlabel('Age (years)')
plt.ylabel('cortex CT (mm)')
```

```
Text(0, 0.5, 'cortex CT (mm)')
```



```
In [ ]: print("Dataframe shape before NaN removal:", df.shape)
```

```
Dataframe shape before NaN removal: (73, 35)
```

Removing missing data

```
In [ ]: df.dropna(axis=0, how='any', inplace=True)
        print("Dataframe shape after NaN removal:", df.shape)
```

```
Dataframe shape after NaN removal: (72, 35)
```

Preparing the design matrix X and the desired output y

```
In [ ]: X = df.iloc[:,2::]
        y = df['Age']
        print (type(X))
        print (type(y))
        print (X.shape)
        print (y.shape)
```

```
<class 'pandas.core.frame.DataFrame'>
<class 'pandas.core.series.Series'>
(72, 33)
(72,)
```

A quick quality control

```
In [ ]: X.head(6)
```

```
In [ ]: print('The whole dataset contains ' + str(X.shape[0]) + ' subjects')
        print('The age prediction will be performed using ' + str(X.shape[1]) + ' MRI-
        derived features')
```

```
The whole dataset contains 72 subjects
The age prediction will be performed using 33 MRI-derived features
```

Preparing data for a classification task

Define a function to binarize age

```
In [ ]: def categorize_age(age): #function implemented to binarize the age
        if age <= 11:
            return 0
        elif age >= 12:
            return 1
```

Binarizing age to obtain two classes

```
In [ ]: df['Age_Category'] = df['Age'].apply(categorize_age) #create a new column in the
        df using the binarized ages
```

```
#df['Age_Category'] = df['Age_Category'].astype(int)
```

```
# Count the occurrences of each unique value in the 'Age_Category' column
age_category_counts = df['Age_Category'].value_counts() #counts how many 0 and 1
appears
```

```
# Display the counts
print("Number of rows with Age_Category equal to 0:", age_category_counts[0])
print("Number of rows with Age_Category equal to 1:", age_category_counts[1])
```

```
Number of rows with Age_Category equal to 0: 32
Number of rows with Age_Category equal to 1: 40
```

A quick quality control

```
In [ ]: df.tail(10)
```

Extracting "Age_category" and removing the "Age", "Sex" columns from the data

```
In [ ]: y = df['Age_Category']
        X = df.drop(columns=['Age_Category','Age','Sex']) #start from the original df it
        cut the listed columns and save it on X
```

```
# Convert the 'Age_Category' column to integer type
#df['Age_Category'] = df['Age_Category'].astype(int)
```

```
print('The whole dataset contains ' + str(X.shape[0]) + ' subjects')
print('The age prediction will be performed using ' + str(X.shape[1]) + ' MRI-
derived features')
```

```
The whole dataset contains 72 subjects
The age prediction will be performed using 33 MRI-derived features
```

A quick quality control

```
In [ ]: X.tail(10)
```

```
In [ ]: y.tail(10)
```

Classification task

Logistic regression using a holdout scheme

```
In [ ]: from sklearn.linear_model import LogisticRegression #import the logistic
        regression algorithm
        from sklearn.model_selection import train_test_split

        # Setting the seed of the random generator
        SEED = 42
        # Size of the samples in the test set: e.g., 0.1, means that the test set is
        composed of 10% of the samples of the entire dataset
        test_size = 0.1

        # Creating the splitter
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=test_size,
        random_state=SEED) #divide the dataset in train and test sets
```

```
In [ ]: print (X_train.shape, y_train.shape)
        print (X_test.shape, y_test.shape)
```

```
(64, 33) (64,)
(8, 33) (8,)
```

```
In [ ]: from sklearn.metrics import roc_auc_score #import the function to compute the AUC

        clf_holdout = LogisticRegression(max_iter=1000) #initialize the classifier of
        LR, specifying max number of iterations
        clf_holdout.fit(X_train, y_train) #trains the model computing the coefficients of
        the line that best divides the two age classes
        y_pred_proba_test = clf_holdout.predict_proba(X_test)[: , 1] #instead of directly
        0/1 this predict a probability for each class on test set, for all data from the 2
        column on
        y_pred_proba_train = clf_holdout.predict_proba(X_train)[: , 1] #same but on
        training set

        auc_train = roc_auc_score(y_train, y_pred_proba_train) #compute AUC comparing real
        labels (train) with predictions (pred_proba_train)
        auc_test = roc_auc_score(y_test, y_pred_proba_test) #=

        print("Average AUC training set:", auc_train)
        print("Average AUC test set:", auc_test)
```

```
Average AUC training set: 0.968627450980392
Average AUC test set: 1.0
```

Logistic Regression with hyperparameter C (Complexity) using a nested holdout scheme

```
In [ ]: #in this block we do a double split, first one we divide the whole dataset in
        development and test, the second we divide the development set in training and
        validation
        from sklearn.model_selection import train_test_split

        # Setting the seed of the random generator
        SEED = 42

        # Size of the samples in the test set: e.g., 0.1, means that the test set is
        composed of 10% of the samples of the entire dataset
        test_size = 0.1

        # Size of the samples in the validation set: e.g., 0.1, means that the test set is
        composed of 10% of the samples of the development set
        val_size = 0.1

        # Outer hold-out
        X_dev, X_test, y_dev, y_test = train_test_split(X, y, test_size=test_size,
        random_state=SEED)

        # Print the size of the development and test sets
        print ("The development set size is", "X:", X_dev.shape, "y:", y_dev.shape)
        print ("The test set size is", "X:", X_test.shape, "y:", y_test.shape)

        # Inner hold-out
        X_train, X_val, y_train, y_val = train_test_split(X_dev, y_dev,
        test_size=val_size, random_state=SEED)
        # Print the size of the training and validation sets
        print ("\nThe development set size is", "X:", X_train.shape, "y:", y_train.shape)
        print ("The test set size is", "X:", X_val.shape, "y:", y_val.shape)
        print()
```

```
The development set size is X: (64, 33) y: (64,)
The test set size is X: (8, 33) y: (8,)
```

```
The development set size is X: (57, 33) y: (57,)
The test set size is X: (7, 33) y: (7,)
```

```
In [ ]: AUC = []

        C_list = [0.1, 1, 10, 100]

        for c in C_list:
            clf = LogisticRegression(max_iter=1000, C=c).fit(X_train, y_train) #initialize
            the linear classifier imposing max iterations e current hyperparameter C, train
            the model on training set computing optimal coefficients
            y_pred_proba_val = clf.predict_proba(X_val)[: , 1] #take the prob from the
            model of the validation set for the positive class (1)
            auc_val = roc_auc_score(y_val, y_pred_proba_val) #compute AUC of the
            validation set confronting real labels with prob just computed
            print("Average AUC val set:", auc_val)
            AUC.append (auc_val) #add the value
        bestC_index = np.argmax(AUC)
        bestC = C_list[bestC_index]
        print ("\nBest C: ", bestC, "AUC", AUC[bestC_index])
```

```
clf = LogisticRegression(max_iter=1000,C=bestC).fit(X_dev, y_dev) #initialize a
new LR model with the best C and train it on the whole development set

# Apply the model to the test set and compute the MAE
y_pred_test = clf.predict_proba(X_test)[:, 1] #generate prob predictions for the
positive class on test set data
auc_test = roc_auc_score(y_test, y_pred_test) #compute final AUC
print("Average AUC test set:", auc_test)
```

```
Average AUC val set: 0.9166666666666667
Average AUC val set: 0.9166666666666667
Average AUC val set: 0.9166666666666667
Average AUC val set: 0.75
```

```
Best C: 0.1 AUC 0.9166666666666667
Average AUC test set: 1.0
```

ROC Curve: Simple Example

In this notebook, we illustrate how to construct a Receiver Operating Characteristic (ROC) curve using a simple dataset of 10 subjects. Five are positive cases (p), and five are negative cases (n). A simulated model assigns a score to each subject, and we examine how changing the threshold affects the classification.

Step 1: Define the dataset

We define two toy datasets:

- `data_ideal` : a perfectly separable scenario where the model assigns very high scores to all positives and low scores to all negatives.
- `data_real` : a more realistic and noisy example with some overlap between positive and negative scores.

You can toggle between the two by assigning either `data_ideal` or `data_real` to the variable `data` .

The data is then loaded into a pandas DataFrame and displayed.

```
In [1]: import pandas as pd

# Define synthetic data: 10 subjects (5 positive, 5 negative)
data_ideal = {
    'Subject': [f'Subject {i+1}' for i in range(10)], #generates a list of strings
    'True Label': ['p', 'p', 'p', 'p', 'p', 'p', 'n', 'n', 'n', 'n'], #postive and
    'Model Score': [0.99999, 0.99999, 0.99993, 0.99986, 0.99964, 0.99955, 0.68139,
    0.50961, 0.48880, 0.44951] #probability given by the model
}

#idem but more real
data_real = {
    'Subject': [f'Subject {i+1}' for i in range(20)],
    'True Label': ['p', 'p', 'n', 'p', 'p', 'p', 'n', 'n', 'p', 'n', 'p', 'n',
    'p', 'n', 'n', 'n', 'p', 'n', 'p', 'n'],
    'Model Score': [0.9, 0.8, 0.7, 0.6, 0.55, 0.54, 0.53, 0.52, 0.51, 0.505, 0.4,
    0.39, 0.38, 0.37, 0.36, 0.35, 0.34, 0.33, 0.30, 0.1]
```

```
}

#data = data_ideal
data = data_real

df = pd.DataFrame(data) #convert the python dictionary in a Pandas df
df
```

Out[1]:

	Subject	True Label	Model Score
0	Subject 1	p	0.900
1	Subject 2	p	0.800
2	Subject 3	n	0.700
3	Subject 4	p	0.600
4	Subject 5	p	0.550
5	Subject 6	p	0.540
6	Subject 7	n	0.530
7	Subject 8	n	0.520
8	Subject 9	p	0.510
9	Subject 10	n	0.505
10	Subject 11	p	0.400
11	Subject 12	n	0.390
12	Subject 13	p	0.380
13	Subject 14	n	0.370
14	Subject 15	n	0.360
15	Subject 16	n	0.350
16	Subject 17	p	0.340
17	Subject 18	n	0.330
18	Subject 19	p	0.300
19	Subject 20	n	0.100

Step 2: Convert labels and compute ROC

We convert the labels to binary (1 = positive, 0 = negative), then compute the ROC curve using `sklearn.metrics.roc_curve` .

```
In [2]: import numpy as np
from sklearn.metrics import confusion_matrix

# Convert labels to binary
y_true = [1 if label == 'p' else 0 for label in df['True Label']] #translates p/n
in 1/0
y_scores = df['Model Score'] #extracts the prob values predicted from the model
from the df

# Use all unique thresholds
all_thresholds = np.sort(np.unique(y_scores))[:-1]
fpr_list = []
tpr_list = []

# Compute initial point for threshold = +infinity (all predicted as 0)
y_pred_inf = [0 for _ in y_scores] # no positives predicted, list of full zeros
the same length as y_scores (very high threshold)
tn, fp, fn, tp = confusion_matrix(y_true, y_pred_inf).ravel() #compute the
confusion matrix confronting real labels with our full 0 vector, ravel is used to
"flatten" the matrix for extracting tn tp fn fp
fpr_list = [fp / (fp + tn)]
tpr_list = [tp / (tp + fn)]
all_thresholds = [np.inf] # start list with infinity(initialize the threshold
list using the positive inf as first value)

# Add the rest of the thresholds
for thresh in np.sort(np.unique(y_scores))[:-1]: #finds all unique scores in the
dataset, then sorts them from smaller threshold to bigger threshold
    y_pred = [1 if score >= thresh else 0 for score in y_scores] #apply the
threshold, if < then 0, if >= then 1
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel() #new confusion
matrix with new threshold
    fpr = fp / (fp + tn) if (fp + tn) != 0 else 0
    tpr = tp / (tp + fn) if (tp + fn) != 0 else 0
    fpr_list.append(fpr) #add values to respective lists (fpr, tpr, thresholds)
    tpr_list.append(tpr)
    all_thresholds.append(thresh)
```

Step 3: Plot the ROC Curve

We now visualize the ROC curve, showing the tradeoff between True Positive Rate (TPR) and False Positive Rate (FPR).

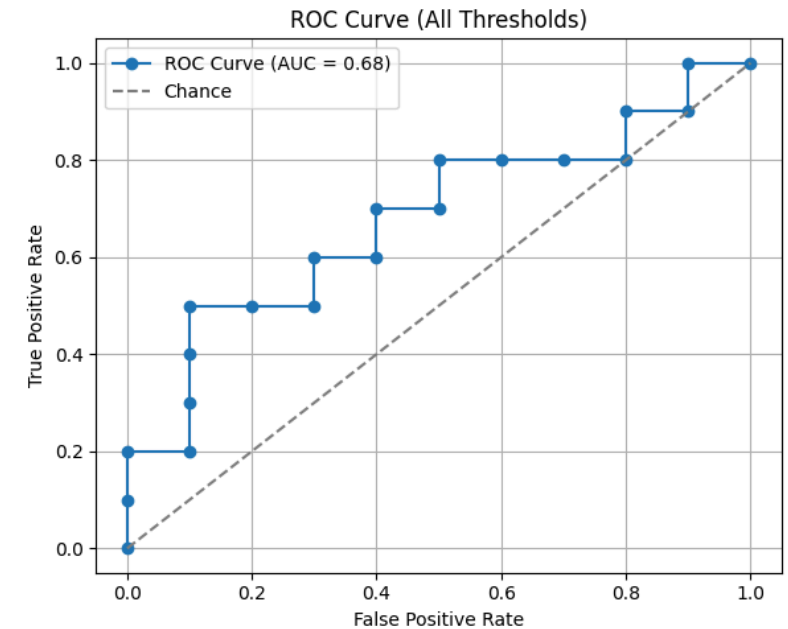
```
In [3]: import matplotlib.pyplot as plt
from sklearn.metrics import auc

roc_auc = auc(fpr_list, tpr_list) #compute AUC using the lists obtained before

plt.figure(figsize=(6, 5))
plt.plot(fpr_list, tpr_list, marker='o', label=f'ROC Curve (AUC = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Chance')
plt.xlabel('False Positive Rate')
```

```
plt.ylabel('True Positive Rate')
plt.title('ROC Curve (All Thresholds)')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
# Create ROC table
roc_table = pd.DataFrame({
    'Threshold': all_thresholds,
    'False Positive Rate': fpr_list,
    'True Positive Rate': tpr_list
})
roc_table
```



	Threshold	False Positive Rate	True Positive Rate
0	inf	0.0	0.0
1	0.900	0.0	0.1
2	0.800	0.0	0.2
3	0.700	0.1	0.2
4	0.600	0.1	0.3
5	0.550	0.1	0.4
6	0.540	0.1	0.5
7	0.530	0.2	0.5

	Threshold	False Positive Rate	True Positive Rate
8	0.520	0.3	0.5
9	0.510	0.3	0.6
10	0.505	0.4	0.6
11	0.400	0.4	0.7
12	0.390	0.5	0.7
13	0.380	0.5	0.8
14	0.370	0.6	0.8
15	0.360	0.7	0.8
16	0.350	0.8	0.8
17	0.340	0.8	0.9
18	0.330	0.9	0.9
19	0.300	0.9	1.0
20	0.100	1.0	1.0

```
print(f"True Positive Rate (TPR): {tpr:.2f}")
print(f"False Positive Rate (FPR): {fpr:.2f}")
print("\nConfusion Matrix:")
print(cm)

# Plot ROC with dynamically computed TPR/FPR point
plt.figure(figsize=(6, 5))
plt.plot(fpr_list, tpr_list, label='ROC Curve')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Chance')
plt.scatter(fpr, tpr, color='red', s=100, label=f'Threshold = {threshold:.2f}')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve with Threshold Marker')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
interactive(children=(FloatSlider(value=0.5, description='threshold',
max=1.0, step=0.05), Output()), _dom_cla...
```

Summary

- The ROC curve is constructed by changing the threshold applied to the model's score.
- Each point on the curve corresponds to a specific threshold.
- The area under the curve (AUC) gives a scalar value representing the model's ability to distinguish between the two classes.
- The interactive slider allows exploration of classification results as a function of threshold.

You can explore how the curve would change by modifying the scores or the number of samples.

Step 4: Explore the effect of threshold interactively

Use the slider below to change the threshold and observe how predictions and confusion metrics change.

```
In [4]: from ipywidgets import interact

#@ = "take the function defined below and link it to an interactive slider"
@interact(threshold=(0.0, 1.0, 0.05)) #thresholds can change between 0.0 and 1.0
with a stepsize of 0.05
def show_confusion(threshold=0.5): #define the function that is called at each
threshold change
    y_pred = [1 if score >= threshold else 0 for score in y_scores]
    cm = confusion_matrix(y_true, y_pred)
    tn, fp, fn, tp = cm.ravel()

    # Compute FPR and TPR manually
    fpr = fp / (fp + tn) if (fp + tn) != 0 else 0
    tpr = tp / (tp + fn) if (tp + fn) != 0 else 0

    # Print results
    print(f"Threshold: {threshold:.2f}")
    print(f"True Positives: {tp}")
    print(f"False Negatives: {fn}")
    print(f"False Positives: {fp}")
    print(f"True Negatives: {tn}")
```

Weight Decay and Overfitting – Interactive Demo

In this demo, we use polynomial regression with a fixed high degree (degree = 25) to explore the effects of weight decay (L2 regularization).

The true function we want to learn is a simple quadratic curve.
We generate noisy observations of that curve.

You can interactively adjust:

- The strength of the weight decay (`lambda` value)
- The noise level added to the data

Use the sliders to observe how regularization helps control overfitting, improves generalization, and stabilizes the model even when using a high-capacity polynomial model.

```
In [ ]: # Install ipywidgets if not already available
!pip install ipywidgets --quiet #library for interactive sliders

from ipywidgets import interact
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import Ridge
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split

# Base data (same for all runs)
np.random.seed(0) #block randomness
n_samples = 100
X = np.linspace(0, 1, n_samples).reshape(-1, 1) #generates 100 points equidistant
from 0 to 1, in column shape
# True underlying function
y_true = 4 * (X.ravel() - 0.5) ** 2 #generates parabola

# Fixed high degree for polynomial expansion
fixed_degree = 25 #thats too high, without regularization it would go overfitting

# Interactive plot function
def plot_weight_decay(lambda_value=0.0, noise_std=0.1): #define the function that
will be executed at each slider change
    # Generate noisy labels
    y = y_true + noise_std * np.random.randn(n_samples) #add the gaussian noise to
the perfect parabola

    # Train/test split
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42) #divide the noisy data in 70% training and 30% test

# Polynomial features
poly = PolynomialFeatures(degree=fixed_degree) #transform the linear instances
into ^25 polynomial, it gives 26 columns for each sample
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

# Train model with Ridge regression (weight decay)
model = Ridge(alpha=lambda_value) # Ridge implements L2 regularization
model.fit(X_train_poly, y_train)

# Predictions and errors, compute the predictions and the relative MSE on
training and test sets
y_train_pred = model.predict(X_train_poly)
y_test_pred = model.predict(X_test_poly)
train_mse = mean_squared_error(y_train, y_train_pred)
test_mse = mean_squared_error(y_test, y_test_pred)

# Plotting
X_plot = np.linspace(0, 1, 200).reshape(-1, 1)
y_plot = model.predict(poly.transform(X_plot))

plt.figure(figsize=(7, 5))
plt.scatter(X_train, y_train, color='blue', label='Train data', s=20)
plt.plot(X_plot, y_plot, color='red', label=f'Degree {fixed_degree}')
plt.plot(X, y_true, '--', color='green', label='True function')
plt.title(f"Train MSE = {train_mse:.4f} | Test MSE = {test_mse:.4f}\nNoise std
= {noise_std} | Lambda = {lambda_value}")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid(True)
plt.show()

# Activate interactive widgets
interact(plot_weight_decay, lambda_value=(0.0, 0.2, 0.05), noise_std=(0.0, 1.0,
0.05))
```

```
interactive(children=(FloatSlider(value=0.0, description='lambda_value',
max=0.2, step=0.05), FloatSlider(valu...
```

```
plot_weight_decay
def plot_weight_decay(lambda_value=0.0, noise_std=0.1)

<no docstring>
```

 MSE vs Weight Decay (Lambda)

This plot shows how the training and test Mean Squared Error (MSE) change as we increase the strength of weight decay (lambda) in a high-degree polynomial model (degree = 25).

We compare two scenarios:

- Low noise (noise_std = 0.15)
- High noise (noise_std = 0.25)

Key insights:

- When lambda = 0 (no weight decay), the model tends to overfit, especially with noisy data.
- Increasing lambda helps reduce overfitting by penalizing large weights, leading to better generalization.
- If lambda becomes too large, the model may underfit and the error increases again.

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import Ridge
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split

# Base data #same as before
np.random.seed(0)
n_samples = 100
X = np.linspace(0, 1, n_samples).reshape(-1, 1)
y_true = 4 * (X.ravel() - 0.5) ** 2

# Settings
fixed_degree = 25
noise_levels = [0.15, 0.25] #different noise levels, low high
lambdas = np.arange(0.0, 0.25, 0.05) #the hyperparameter changes

plt.figure(figsize=(12, 6))

for noise_std in noise_levels: #compute everything firstly with low noise, then
    with high
        y = y_true + noise_std * np.random.randn(n_samples) #add gaussian noise to
        perfect parabola
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
        random_state=42) #divide

        train_mse = []
        test_mse = []

        for lambda_value in lambdas: #inner cicle that tries every single lambda
        available (different weight decays)
            # Polynomial features
            poly = PolynomialFeatures(fixed_degree)
            X_train_poly = poly.fit_transform(X_train)
            X_test_poly = poly.transform(X_test)

            # Model with Ridge (weight decay)
            model = Ridge(alpha=lambda_value)
            model.fit(X_train_poly, y_train) #trains the ridge model finding
            coefficients that minimize the diff from the ral data added to the L2
            penalties on the weights
```

```
y_train_pred = model.predict(X_train_poly) #predictions generation
y_test_pred = model.predict(X_test_poly)

train_mse.append(mean_squared_error(y_train, y_train_pred)) #compute MSEs
and add to relative lists
test_mse.append(mean_squared_error(y_test, y_test_pred))

# Plot for current noise level
plt.plot(lambdas, train_mse, linestyle='--', marker='o', label=f'Train MSE
(noise={noise_std})')
plt.plot(lambdas, test_mse, marker='o', label=f'Test MSE (noise={noise_std})')

plt.xlabel('Lambda (Weight Decay Strength)')
plt.ylabel('MSE')
plt.title('Training and Test MSE vs Weight Decay')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



Age Prediction: data preparation, Exploratory Data Analysis (EDA), and regression with cross-validation (CV) & nested CV

Data Preparation and Exploratory Data Analysis (EDA)

Import libraries

```
In [ ]: # Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Loading and showing input (features) and output data (desired output)

```
In [ ]: df = pd.read_excel('https://raw.githubusercontent.com/sdiciotti/Age-Prediction-
Demo/main/NKI2_data.xlsx')
print (type(df))
```

```
<class 'pandas.core.frame.DataFrame'>
```

Print the size of the dataframe

```
In [ ]: df.shape
```

```
(73, 35)
```

Print the dataframe

```
In [ ]: print(df)
```

Check the features and output variables name

```
In [ ]: features = list(df.columns)
type(features)
```

```
list
```

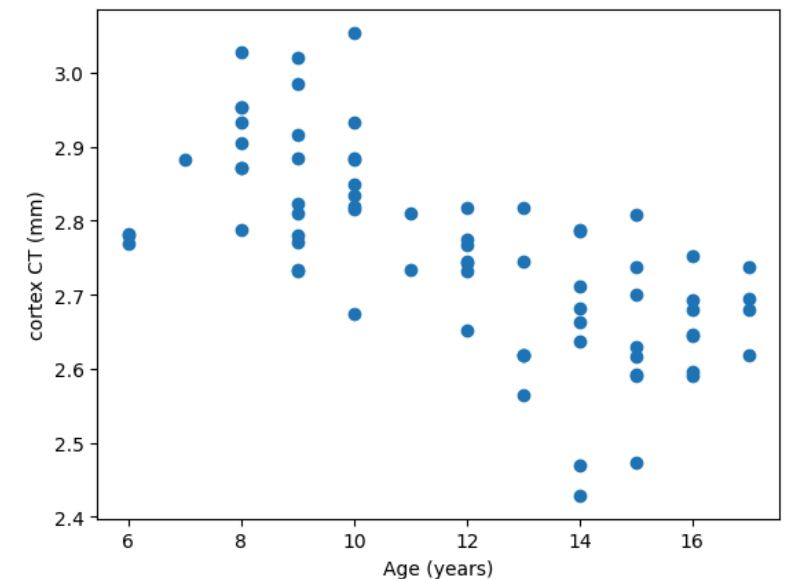
```
In [ ]: print(features)
```

```
['Age', 'Sex', 'cortex_CT', 'lh_cortex_CT', 'rh_cortex_CT',
'lh_frontalGM_CT', 'lh_occipitalGM_CT', 'lh_temporalGM_CT',
'lh_parietalGM_CT', 'rh_frontalGM_CT', 'rh_occipitalGM_CT',
'rh_temporalGM_CT', 'rh_parietalGM_CT', 'cortex_pial_lgi',
'lh_cortex_pial_lgi', 'rh_cortex_pial_lgi', 'lh_frontal_pial_lgi',
'lh_occipital_pial_lgi', 'lh_temporal_pial_lgi', 'lh_parietal_pial_lgi',
'rh_frontal_pial_lgi', 'rh_occipital_pial_lgi', 'rh_temporal_pial_lgi',
'rh_parietal_pial_lgi', 'cerebralGM_FD', 'lh_cerebralGM_FD',
'lh_frontalGM_FD', 'lh_temporalGM_FD', 'lh_parietalGM_FD',
'lh_occipitalGM_FD', 'rh_cerebralGM_FD', 'rh_frontalGM_FD',
'rh_temporalGM_FD', 'rh_parietalGM_FD', 'rh_occipitalGM_FD']
```

Plotting some data

```
In [ ]: # Plot true vs predicted values to visualize model performance
plt.scatter(df['Age'],df['cortex_CT'])
plt.xlabel('Age (years)')
plt.ylabel('cortex CT (mm)')
```

```
Text(0, 0.5, 'cortex CT (mm)')
```



```
In [ ]: print("Dataframe shape before NaN removal:", df.shape)
```

```
Dataframe shape before NaN removal: (73, 35)
```

Removing missing data

```
In [ ]: df.dropna(axis=0, how='any', inplace=True)
print("Dataframe shape after NaN removal:", df.shape)
```

Dataframe shape after NaN removal: (72, 35)

Preparing the design matrix X and the desired output y

```
In [ ]: X = df.iloc[:,2::]
y = df['Age']
print (type(X))
print (type(y))
print (X.shape)
print (y.shape)
```

```
<class 'pandas.core.frame.DataFrame'>
<class 'pandas.core.series.Series'>
(72, 33)
(72,)
```

A quick quality control

```
In [ ]: X.head(6)
```

```
In [ ]: print('The whole dataset contains ' + str(X.shape[0]) + ' subjects')
print('The age prediction will be performed using ' + str(X.shape[1]) + ' MRI-
derived features')
```

The whole dataset contains 72 subjects
The age prediction will be performed using 33 MRI-derived features

Regression task

Linear regression using a CV scheme

```
In [ ]: from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_validate, KFold #import tools
from sklearn.svm import SVR
```

```
# Setting the seed of the random generator
SEED = 42
```

```
# Setting the number of folds
```

```
n_folds = 5 #k
```

```
# Creating the estimator
#reg = LinearRegression()
reg = SVR(kernel='rbf', degree=3, gamma='scale', coef0=0.0, tol=0.001, C=1,
epsilon=0.1, shrinking=True, cache_size=200, verbose=0, max_iter=- 1) #define the
final estimator model
```

```
# Creating the splitter
cv = KFold(n_splits=n_folds, shuffle=True, random_state=SEED) #this will organize
the k splits, shuffle active means that before splitting we shuffle the folds to
prevent pattern
```

```
In [ ]: # Print the generated splits
for train_index, test_index in cv.split(X): #takes as input X and produce folds,
then returns the two lists (train and test), this is done at every iteration of
the cycle
    print("Train:", train_index, " Test:", test_index)
```

```
In [ ]: score = cross_validate(reg, X=X, y=y, cv=cv, return_train_score=True, scoring =
'neg_mean_absolute_error') #does CV, neg perchè di base cross_validate da come
migliore il valore piu alto, vogliamo l'inverso

print("This is the score object:")
print (score)

print("Average MAE training set:", np.mean(np.abs(score['train_score'])), "years")
print("Average MAE tes set:", np.mean(np.abs(score['test_score'])), "years")
```

```
This is the score object:
{'fit_time': array([0.03309894, 0.00383759, 0.01326203, 0.0036819 ,
0.00364852]), 'score_time': array([0.01271653, 0.00284362, 0.00788593,
0.00270724, 0.00272942]), 'test_score': array([-1.96516495, -1.69834222,
-1.47619532, -1.3528025 , -2.07928787]), 'train_score': array([-1.51757213,
-1.72181049, -1.69668166, -1.70997242, -1.56123815])}
Average MAE training set: 1.6414549705947792 years
Average MAE tes set: 1.7143585706462816 years
```

```
In [ ]: print(type(score))
print(type(score['train_score']))
```

```
<class 'dict'>
<class 'numpy.ndarray'>
```

Support Vector Regression with hyperparameter C (Complexity) using a nested CV scheme

```
In [ ]: from sklearn.model_selection import train_test_split, KFold, GridSearchCV,
cross_validate
from sklearn.svm import SVR
from sklearn.metrics import mean_absolute_error
```

```

# Setting the seed of the random generator
SEED = 42

# Setting the number of folds of both outer and inner k-fold CV
outer_n_folds = 5
inner_n_folds = 5

# Setting the possible values of the C hyperparameter
C = [0.1, 1, 10]

# Creating the splitters
outer_cv = KFold(n_splits=outer_n_folds, shuffle=True, random_state=SEED)
inner_cv = KFold(n_splits=inner_n_folds, shuffle=True, random_state=SEED)

# Creating the estimator
reg = SVR(kernel='rbf', degree=3, gamma='scale', coef0=0.0, tol=0.001, C=0.1,
epsilon=0.1, shrinking=True, cache_size=200, verbose=0, max_iter=-1)

# Defining the grid of hyperparameter values
p_grid = [{'C': C}]

reg_gs = GridSearchCV(reg, param_grid=p_grid, cv=inner_cv,
refit='neg_mean_absolute_error', scoring='neg_mean_absolute_error', verbose = 4)
nested_score = cross_validate(reg_gs, X=X, y=y, cv=outer_cv,
return_train_score=True, return_estimator=True, scoring =
'neg_mean_absolute_error')

#print(np.abs(nested_score['train_score']))
#print(np.abs(nested_score['test_score']))
print("Average MAE train:", np.abs(np.mean(nested_score['train_score'])), "years")
print("Average MAE test:", np.abs(np.mean(nested_score['test_score'])), "years")

```

```

In [ ]: print(type(p_grid))
        print(type(p_grid[0]))

```

```

<class 'list'>
<class 'dict'>

```

Age Prediction: data preparation, Exploratory Data Analysis (EDA), and regression with cross-validation (CV) & nested CV

Data Preparation and Exploratory Data Analysis (EDA)

```
In [ ]: #here everything is the same as the last one but before all the computation we
        need the threshold etc.
```

Import libraries

```
In [ ]: # Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Loading and showing input (features) and output data (desired output)

```
In [ ]: df = pd.read_excel('https://raw.githubusercontent.com/sdiciotti/Age-Prediction-
        Demo/main/NKI2_data.xlsx')
print (type(df))
```

```
<class 'pandas.core.frame.DataFrame'>
```

Print the size of the dataframe

```
In [ ]: df.shape
```

```
(73, 35)
```

Print the dataframe

```
In [ ]: print(df)
```

Check the features and output variables name

```
In [ ]: features = list(df.columns)
        type(features)
```

```
list
```

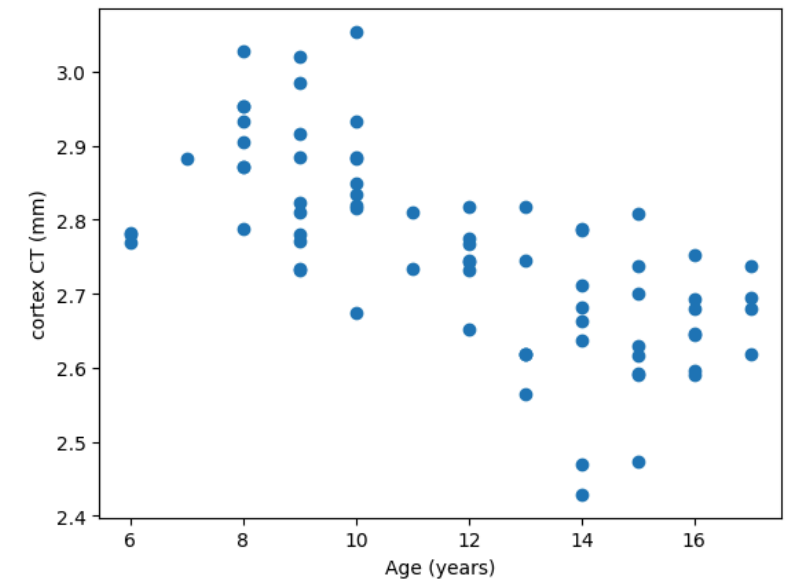
```
In [ ]: print(features)
```

```
['Age', 'Sex', 'cortex_CT', 'lh_cortex_CT', 'rh_cortex_CT',
 'lh_frontalGM_CT', 'lh_occipitalGM_CT', 'lh_temporalGM_CT',
 'lh_parietalGM_CT', 'rh_frontalGM_CT', 'rh_occipitalGM_CT',
 'rh_temporalGM_CT', 'rh_parietalGM_CT', 'cortex_pial_lgi',
 'lh_cortex_pial_lgi', 'rh_cortex_pial_lgi', 'lh_frontal_pial_lgi',
 'lh_occipital_pial_lgi', 'lh_temporal_pial_lgi', 'lh_parietal_pial_lgi',
 'rh_frontal_pial_lgi', 'rh_occipital_pial_lgi', 'rh_temporal_pial_lgi',
 'rh_parietal_pial_lgi', 'cerebralGM_FD', 'lh_cerebralGM_FD',
 'lh_frontalGM_FD', 'lh_temporalGM_FD', 'lh_parietalGM_FD',
 'lh_occipitalGM_FD', 'rh_cerebralGM_FD', 'rh_frontalGM_FD',
 'rh_temporalGM_FD', 'rh_parietalGM_FD', 'rh_occipitalGM_FD']
```

Plotting some data

```
In [ ]: # Plot true vs predicted values to visualize model performance
plt.scatter(df['Age'],df['cortex_CT'])
plt.xlabel('Age (years)')
plt.ylabel('cortex CT (mm)')
```

```
Text(0, 0.5, 'cortex CT (mm)')
```



```
In [ ]: print("Dataframe shape before NaN removal:", df.shape)
```

```
Dataframe shape before NaN removal: (73, 35)
```

Removing missing data

```
In [ ]: df.dropna(axis=0, how='any', inplace=True)
print("Dataframe shape after NaN removal:", df.shape)
```

```
Dataframe shape after NaN removal: (72, 35)
```

Preparing the design matrix X and the desired output y

```
In [ ]: X = df.iloc[:,2::]
y = df['Age']
print (type(X))
print (type(y))
print (X.shape)
print (y.shape)
```

```
<class 'pandas.core.frame.DataFrame'>
<class 'pandas.core.series.Series'>
(72, 33)
(72,)
```

A quick quality control

```
In [ ]: X.head(6)
```

```
In [ ]: print('The whole dataset contains ' + str(X.shape[0]) + ' subjects')
print('The age prediction will be performed using ' + str(X.shape[1]) + ' MRI-
derived features')
```

```
The whole dataset contains 72 subjects
The age prediction will be performed using 33 MRI-derived features
```

Preparing data for a classification task

Define a function to binarize age

```
In [ ]: def categorize_age(age):
    if age <= 11:
        return 0
    elif age >= 12:
        return 1
```

Binarizing age to obtain two classes

```
In [ ]: df['Age_Category'] = df['Age'].apply(categorize_age)

#df['Age_Category'] = df['Age_Category'].astype(int)

# Count the occurrences of each unique value in the 'Age_Category' column
age_category_counts = df['Age_Category'].value_counts()

# Display the counts
print("Number of rows with Age_Category equal to 0:", age_category_counts[0])
print("Number of rows with Age_Category equal to 1:", age_category_counts[1])
```

```
Number of rows with Age_Category equal to 0: 32
Number of rows with Age_Category equal to 1: 40
```

A quick quality control

```
In [ ]: df.tail(10)
```

Extracting "Age_category" and removing the "Age", "Sex" columns from the data

```
In [ ]: y = df['Age_Category']
X = df.drop(columns=['Age_Category', 'Age', 'Sex'])

# Convert the 'Age_Category' column to integer type
#df['Age_Category'] = df['Age_Category'].astype(int)

print('The whole dataset contains ' + str(X.shape[0]) + ' subjects')
print('The age prediction will be performed using ' + str(X.shape[1]) + ' MRI-
derived features')
```

```
The whole dataset contains 72 subjects
The age prediction will be performed using 33 MRI-derived features
```

A quick quality control

```
In [ ]: X.tail(10)
```

```
In [ ]: y.tail(10)
```

Classification task

Logistic regression using a CV scheme

```
In [ ]: from sklearn.linear_model import LinearRegression
        from sklearn.model_selection import cross_validate, KFold
        from sklearn.linear_model import LogisticRegression

        # Setting the seed of the random generator
        SEED = 42

        # Setting the number of folds
        n_folds = 5

        # Creating the estimator
        clf = LogisticRegression()

        # Creating the splitter
        cv = KFold(n_splits=n_folds, shuffle=True, random_state=SEED)

In [ ]: # Print the generated splits
        for train_index, test_index in cv.split(X):
            print("Train:", train_index, " Test:", test_index)

In [ ]: score = cross_validate(clf, X=X, y=y, cv=cv, return_train_score=True,
                               return_estimator=True, scoring = 'roc_auc')

        print("This is the score object:")
        print(score)

        print("Average AUC training set:", np.mean(score['train_score']))
        print("Average AUC test set:", np.mean(score['test_score']))

        This is the score object:
        {'fit_time': array([0.01678705, 0.00947547, 0.00753498, 0.0042944 ,
        0.00671983]), 'score_time': array([0.00604224, 0.00256777, 0.00238848,
        0.00238776, 0.00235534]), 'estimator': [LogisticRegression(),
        LogisticRegression(), LogisticRegression(),
        LogisticRegression()], 'test_score': array([0.90740741, 1.
        , 1.
        , 1.
        , 0.91666667]), 'train_score': array([0.98014888, 0.96464646,
        0.96848485, 0.96415771, 0.98317308])}
        Average AUC training set: 0.9721221959044539
        Average AUC test set: 0.9648148148148149

In [ ]: print(type(score))
        print(type(score['train_score']))

        <class 'dict'>
        <class 'numpy.ndarray'>
```

Logistic Regression with hyperparameter C (Complexity) using a nested CV scheme

```
In [ ]: from sklearn.model_selection import train_test_split, KFold, GridSearchCV,
        cross_validate
        from sklearn.linear_model import LogisticRegression
        from sklearn.metrics import mean_absolute_error

        # Setting the seed of the random generator
        SEED = 42

        # Setting the number of folds of both outer and inner k-fold CV
        outer_n_folds = 5
        inner_n_folds = 5

        # Setting the possible values of the C hyperparameter
        C = [0.1, 1, 10]

        # Creating the splitters
        outer_cv = KFold(n_splits=outer_n_folds, shuffle=True, random_state=SEED)
        inner_cv = KFold(n_splits=inner_n_folds, shuffle=True, random_state=SEED)

        # Creating the estimator
        clf = LogisticRegression(max_iter=1000)

        # Defining the grid of hyperparameter values
        p_grid = [{'C': C}]

        clf_gs = GridSearchCV(clf, param_grid=p_grid, cv=inner_cv, refit='roc_auc',
                               scoring='roc_auc', verbose = 4)
        nested_score = cross_validate(clf_gs, X=X, y=y, cv=outer_cv,
                                       return_train_score=True, return_estimator=True, scoring = 'roc_auc')

        #print(np.abs(nested_score['train_score']))
        #print(np.abs(nested_score['test_score']))
        print("Average MAE train:", np.mean(np.abs(nested_score['train_score'])), "years")
        print("Average MAE test:", np.mean(np.abs(nested_score['test_score'])), "years")

In [ ]: print(type(p_grid))
        print(type(p_grid[0]))

        <class 'list'>
        <class 'dict'>
```